Git，从零开始

本文假设你从未输入过 Git 命令。如果你已经了解提交和分支的概念，请直接浏览 Gitea 相关部分和合并冲突处理流程，然后停止阅读。

在项目 1 截止日期之前阅读此内容——在本课程中，如果不提交，就无法提交任何内容， `git push` 如果你还不知道什么是提交，那么提交的内容就毫无意义。

这一切存在的意义是什么？

如果没有版本控制，“我的项目”就只是磁盘上当前存储的内容。如果你搞砸了，你唯一能追溯到的历史记录就是你记得的部分。Git 会保存你指定跟踪的每个文件的每个版本的完整历史记录，并带有时间戳，因此你总能回滚到之前可以正常运行的版本——这样，两个人（你的 P2 合作伙伴，你的 P3 团队）就可以在同一个项目上工作，而不会互相覆盖彼此的更改。

本课程使用Gitea，而非 GitHub。课程代码仓库托管在自托管的 Gitea 实例上，而非 GitHub。Gitea 使用相同的 `git` 协议，在浏览器中的界面也类似，因此以下所有操作都完全相同——只有网址不同。您的老师会提供该网址。

在其他一切都变得有意义之前，你只需要四个字。

单词	它的实际含义
存储库（“repo”）	Git 正在跟踪的项目文件夹。 `.git` 该文件夹内有一个隐藏文件夹作为标记——切勿手动编辑该隐藏文件夹。
工作目录	磁盘上的实际文件，就像你现在在 VS Code 中看到的那样。
集结区	一个用于暂存即将保存的更改的区域。 `git add` 在确认保存之前，您可以选择要添加的内容。
犯罪	保存的暂存区快照，并附有解释原因的消息。这是历史记录的单位，而不是“文件”或*提交*。

代码库的历史记录就是一系列提交，每个提交都是一个快照，外加指向前一个提交的指针。

一次性设置

如果您已按照步骤 4 操作，则已完成。 `docs/SETUP_WSL2.md` 否则：

```
git config --global user.name  "Your Name"
git config --global user.email "your.email@university.edu"
git config --global init.defaultBranch main
```

Solo 工作流程：状态 → 添加 → 提交 → 日志

`cpp-course-starter/` 在进行一些更改后，在内部（或你正在进行的任何项目中）执行此操作。

```
git status
```

显示自上次提交以来发生的变化：已修改的文件、git 尚未发现的新文件（“未跟踪”）以及已暂存的文件。**请持续运行此命令。**这是最有用的 git 命令，而且它不会更改任何内容——运行此命令不会破坏任何东西 `git status` 。

```
git add src/grid.cpp
```

暂存该文件——将其放入暂存区，等待下次提交。要暂存所有更改： `git add -A` 。 `git status` 再次运行，您将看到它从“未暂存的更改”变为“待提交的更改”。

```
git commit -m "Fix off-by-one in Grid::row"
```

它会对当前所有已准备就绪的内容进行快照，并附上该消息。 **请撰写解释*原因的*消息，而不是描述内容**—— `git log` 它只会显示发生了哪些变化；但它无法告诉你你为什么认为这是一个好主意。

```
git log --oneline
```

显示提交历史记录，最新提交在前。 `git log` （无 `--oneline` ）显示每个提交的完整消息、作者和日期。

每个想法提交一次。五个小的、标签清晰的提交（例如“添加看板结构”、“添加洪水填充显示”、“修复显示递归基本情况”）远比晚上 11:58 的一次“完成项目”的大提交更有用——对你、你的合作伙伴、评分者来说都是如此。

使用 Gitea：克隆、推送、拉取

克隆——获取Gitea上已存在的仓库的副本（您在 `docs/SETUP_WSL2.md` 步骤5中已完成此操作）：

```
git clone https://git.pdulab.net/pdu/2440099 starter
```

这会下载整个历史记录，而不仅仅是最新的快照，并记住它的来源（称为 `origin` ）。

推送——将你本地的提交上传到 Gitea。这*是*本课程的提交机制——就评分而言，你未推送的内容都无效：

```
git push
```

拉取——将其他人（你的 P2 伙伴、你的 P3 队友）推送的提交合并到你自己的工作中：

```
git pull
```

`git pull` *在开始工作会话之前*，以及在推送之前（如果距离上次拉取已过去一段时间），如果您并非完全独自工作，请务必运行此命令。当本地副本落后于 Git 时，推送操作 `origin` 会故意失败——Git 不会默默地丢弃您尚未看到的提交。

分支——适用于 P2 和 P3，但不适用于 P1。

`main` 分支是一条独立的提交记录，在合并回主分支之前不会产生任何影响。其目的是：在不影响其他用户正在使用的工作版本的情况下进行尝试。

```
git branch                          # list branches, * marks the current one
git switch -c try-heap-board         # create AND switch to a new branch
git switch main                      # switch back
git merge try-heap-board             # from main, bring those commits in
```

对于个人项目（P1），您可以完全忽略分支，直接提交到主分支 `main`。对于 P2（两人合作）和 P3（团队合作），常见的模式是：每个人为自己的功能创建分支，推送分支，并在功能正常后合并到 `main` 主分支——这样，即使实验尚未完成，也不会影响到伙伴正在开发的代码。

故意制造合并冲突

这事迟早会发生在你身上，而且通常会在不好的时候发生，所以现在故意制造一个——就像 Lab 0 让你故意制造一份消毒剂报告一样——意味着当它第一次真正发生时，你已经知道你面对的是什么了。

1. 确保 `main` 已提交且干净（`git status` 显示没有任何要提交的内容）。
2. 在两个不同的分支中，用两种不同的方式编辑文件的**同一行：**

```
git switch -c branch-a
# edit README.md, change the first line to "Hello from A"
git add README.md && git commit -m "Branch A edit"

git switch main
git switch -c branch-b
# edit README.md, change that SAME first line to "Hello from B"
git add README.md && git commit -m "Branch B edit"

git switch main
git merge branch-a      # succeeds, no conflict yet
git merge branch-b      # CONFLICT — same line, two different edits
```

3. Git 会停止并标记该文件。打开它——你会看到类似这样的内容：

```
<<<<<<< HEAD
Hello from A
=======
Hello from B
>>>>>>> branch-b
```

介于两者之间的所有内容 `<<<<<<< HEAD` 都是 `=======` 你当前分支上已有的内容。介于两者之间的所有内容 `=======` 则 `>>>>>>> branch-b` 是你要合并到的分支所需要的。Git 无法猜测你想要哪个——这就是它停止运行的原因。

4. **手动解决**：删除逗号 `<<<<<<<` 、逗号 `=======` 和 `>>>>>>>` 标记线，以及你不需要的那一侧，使文件恢复到正确的格式。（如果这样更合理，你也可以保留两边，将它们合并在一起。）
5. 暂存并提交解决方案——合并冲突通过普通提交解决，没有特殊的"冲突提交"命令：

```
git add README.md
git commit -m "Merge branch-b, keep the B greeting"
```

6. 清理练习分支，你不需要它们：

```
git branch -d branch-a branch-b
```

每个人第一次真正遇到合并问题时都会犯两个错误：一是惊慌失措地删除文件重新开始（这样会丢失双方的工作——只需读取标记即可）；二是忘记第四步——提交文件时 *仍然保留着* `<<<<<<<` 标记，这不仅没有修复任何问题，反而会悄无声息地损坏文件。 `git status` 合并工具会提示合并未解决；在提示解决之前不要提交。

常见错误

- **正在提交生成的文件**（ `build/` 编译后的二进制文件）。此仓库 `.gitignore` 已排除 `build/` 某些文件——请勿尝试更改或强行提交。
- `git add` 如果在提交之前**忘记创建一个新文件** `git status` ， 就会显示为未跟踪；这就是为什么要先检查的原因。
- **在提交信息中写"已修复问题"？**六个月后（或者六小时后，在代码答辩中），这毫无意义。
- **提交密钥。**切勿提交 API 密钥，包括 DeepSeek 密钥——参见 `docs/SETUP_WSL2.md` 步骤 7。如果确实提交了密钥，请将其视为已泄露（重新生成密钥），而不是以为之后删除文件就能将其从历史记录中移除——事实并非如此；之前的提交记录中仍然包含该密钥。
- **连续工作数小时却不提交代码。**如果你的编辑器现在崩溃了，你会损失多少代码？这大致就是你应该提交代码的频率。

快速参考

命令	它的作用
`git status`	哪些方面发生了变化，哪些方面是人为安排的——持续关注这个问题
`git add <file>` / `git add -A`	暂存文件/暂存所有内容
`git commit -m "message"`	拍摄已布置好的场景
`git log --oneline`	历史，最新优先
`git clone <url>`	获取一份 Gitea 仓库副本
`git push`	请将您的提交内容发送到 Gitea——这就是提交方式。
`git pull`	降低队友的承诺
`git branch`	列出分支
`git switch -c <name>`	创建并切换到新分支
`git switch <name>`	切换到现有分支
`git merge <name>`	将另一个分支的提交合并到当前分支。
`git diff`	逐行显示未暂存的更改